

Urban Mobility Data Explorer

Technical Report | NYC Yellow Taxi 2025

1. Problem Framing and Dataset Analysis

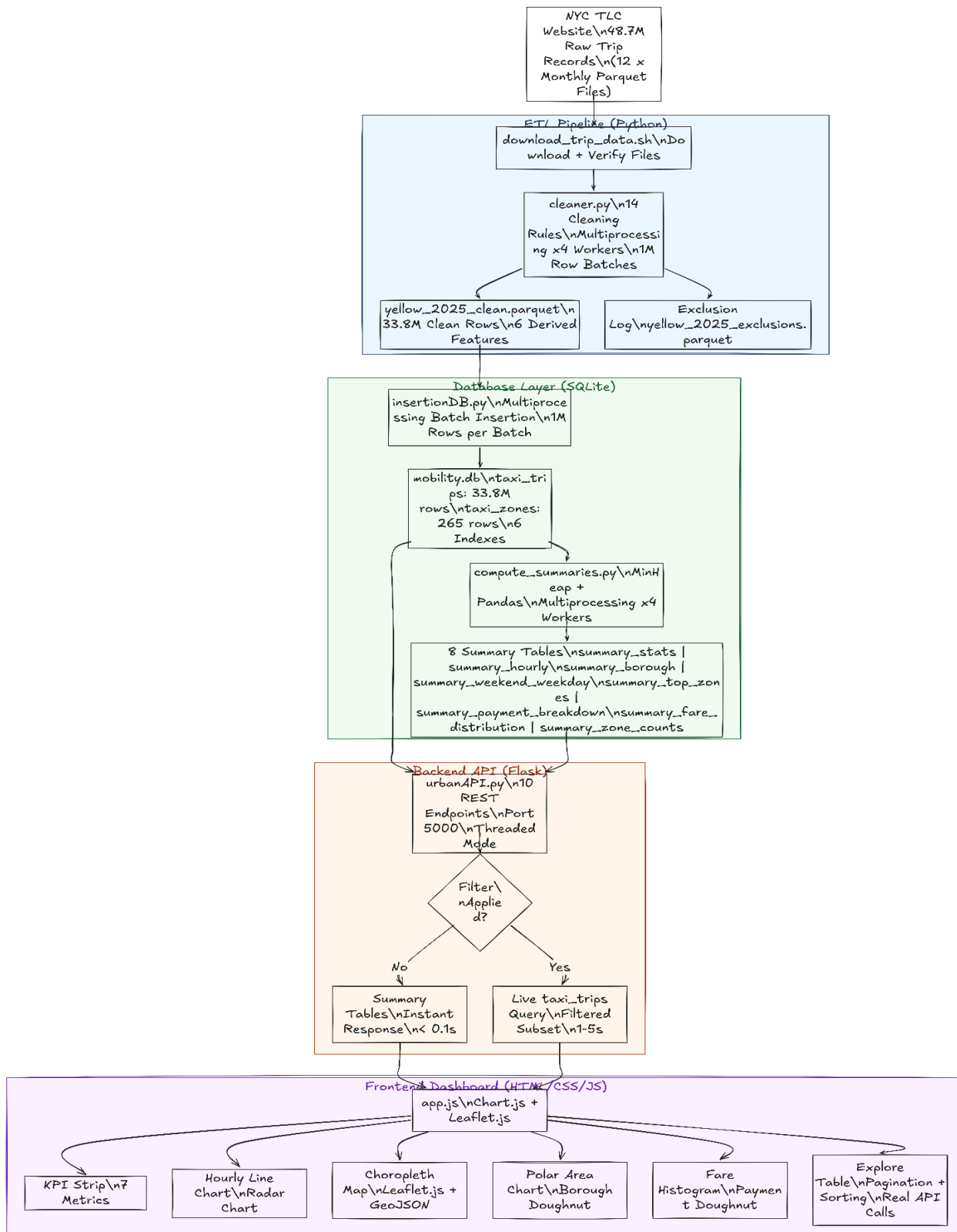
This project processes 48.7 million NYC TLC Yellow Taxi trip records for 2025. Three data components were integrated: the monthly trip Parquet files (fact table), taxi_zone_lookup.csv (dimension table), and taxi_zones GeoJSON (spatial metadata for 263 zones).

Key data challenges identified and resolved:

- **Negative fares:** 2.8 million records contained negative fare_amount values and were excluded.
- **Invalid durations:** Trips where dropoff occurred before pickup were removed. Invalid durations grew from 2,000 in January to 68,000 in June, indicating a systemic data quality issue.
- **Outlier distances and speeds:** Trips beyond 150 miles or computed speeds above 200 mph were treated as sensor errors.
- **Duplicates:** NumPy boolean mask deduplication on 5 key fields removed 721,777 duplicates that a string-based approach missed.

A total of 14 cleaning rules were applied with per-row exclusion logging. The cleaned dataset retains 33,858,070 records (69.5% retention). The unexpected growth pattern in invalid durations influenced the decision to use strict per-row exclusion rather than imputation.

2. System Architecture and Design Decisions



The system follows a four-layer pipeline: ETL (`cleaner.py`) produces a clean Parquet file, `insertionDB.py` loads it into SQLite, `compute_summaries.py` pre-computes 8 summary tables, and `urbanAPI.py` serves 10 REST endpoints to the frontend dashboard.

Stack and Trade-offs

SQLite was chosen over PostgreSQL for portability and zero server setup. Since the dataset is written once and read many times, SQLite's single-writer limitation is not a constraint in practice. Flask was chosen for its minimal configuration and native Python integration.

The most significant architectural decision was pre-computation. Live aggregation queries on 33.8 million rows took 57 to 201 seconds per request in SQLite. By running all aggregations once using the multiprocessing pipeline and storing results in 8 summary tables, every static API endpoint now responds in under 0.1 seconds. Filtered requests query the live table directly, which is fast because WHERE clauses reduce the rows scanned.

Schema (3NF)



Two core tables: taxi_zones (265 rows, stores location_id, borough, zone_name, service_zone) and taxi_trips (33.8M rows, stores all trip fields plus 6 derived features). Borough and zone names are stored only in taxi_zones and referenced by foreign key, eliminating repeated strings across millions of rows.

Six indexes: `idx_pickup_datetime`, `idx_pu_location_id`, `idx_do_location_id`, `idx_time_of_day`, `idx_is_weekend`, `idx_payment_type`.

Eight summary tables: `summary_stats` (1 row, global KPIs), `summary_hourly` (24 rows, per-hour aggregations), `summary_borough` (6 rows), `summary_weekend_weekday` (2 rows), `summary_top_zones` (15 rows, MinHeap output), `summary_payment_breakdown` (5 rows), `summary_fare_distribution` (8 rows), `summary_zone_counts` (260 rows, used by the GeoJSON map).

3. Algorithmic Logic — Custom MinHeap

A MinHeap was implemented in `zone_rank.py` to find the top 15 busiest pickup zones from 263 zones, without using `heapq`, `sort_values`, or any built-in sorting library.

The heap maintains only $k = 15$ items. For each zone, if the heap is not full, insert and `bubble_up`. If the heap is full and the new zone has more trips than the current minimum (root), replace the root and `bubble_down`. Extract and sort the final k items in descending order using a manual bubble sort.

- **Time complexity:** $O(n \log k)$ where $n = 263$ zones, $k = 15$. Approximately 1,052 operations versus 2,104 for full sorting.
- **Space complexity:** $O(k)$. Only 15 items held in memory at any time.

The algorithm runs during `compute_summaries.py` to build `summary_top_zones`, and again at query time when borough or time-of-day filters are applied.

4. Insights and Interpretation

Insight 1: When Does NYC Move?

Trip volume peaks at 18:00 (6 PM) with over 2.4 million trips at that hour in 2025. Average speed drops from 20 mph at 4 AM to below 10 mph by 8 AM as congestion builds, and remains low throughout the working day. The busiest hour is also the slowest, confirming that peak taxi demand coincides with peak congestion. NYC's 2025 central business district congestion pricing, which took effect January 5, may contribute to elevated fares during these peak hours.

When Does NYC Move?

Trip volume and average speed across every hour of the day

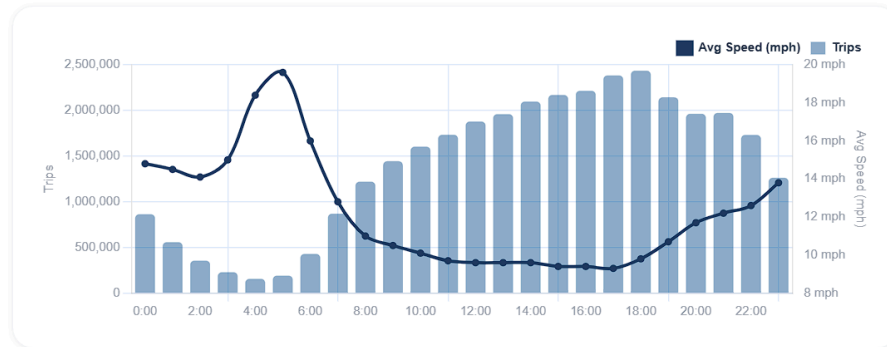
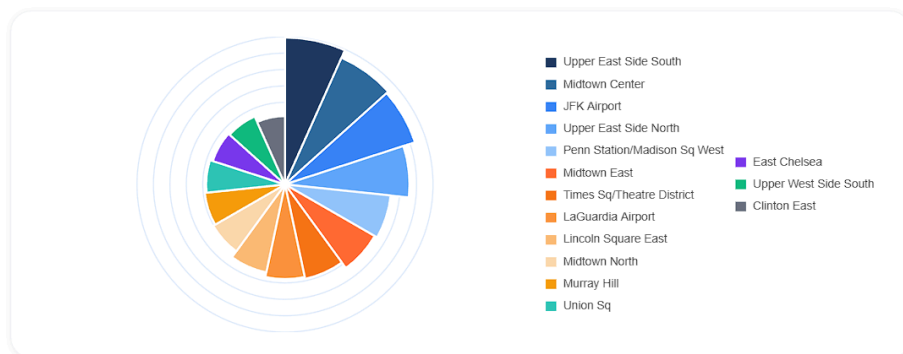


Figure 1: Hourly trip volume (bars) and average speed (line) — full year 2025

Insight 2: The Manhattan Monopoly

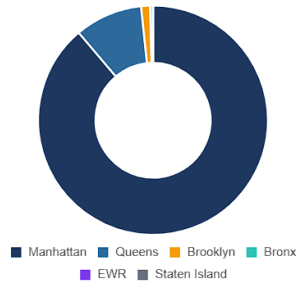
Manhattan accounts for 89% of all yellow taxi pickups in 2025. Thirteen of the top 15 busiest pickup zones are in Manhattan. The only non-Manhattan zones in the top 15 are JFK Airport (rank 3) and LaGuardia Airport (rank 8) in Queens. This reflects the structural reality of yellow taxi licensing, which primarily covers street hails in Manhattan and the airports.

Top 15 busiest pickup zones



The busiest pickup zone is **Upper East Side South** (Manhattan) with **1,790,526** pickups in 2025. **13 of the top 15** zones are in Manhattan, confirming its dominance in yellow taxi activity.

Trip share by borough



Manhattan dominates yellow taxi pickups with **88.8%** of all 2025 trips. **Queens** is a distant second at **9.5%**. The outer boroughs rely far less on yellow taxis, reflecting the city's transit geography.

Figure 2: Borough doughnut and top 15 zones polar area chart

Insight 3: Weekday vs Weekend Patterns

Weekdays account for 24.9 million trips versus 9.0 million on weekends. Despite lower weekend volume, weekend trips are faster by 1.4 mph due to lighter traffic. Weekday fares are \$0.32 higher on average, consistent with longer durations caused by rush hour congestion. Tip percentage is nearly identical across both day types, suggesting tipping behaviour is not influenced by the day of the week.

Weekday vs Weekend, how the city moves differently

WEEKDAY TRIPS
24,872,784

WEEKEND TRIPS
8,985,286

Weekends are faster by **1.4 mph** on average, likely due to lighter traffic. **Weekday** trips cost more on average, with a \$0.32 fare difference. Weekend trips account for **26.5%** of all 2025 taxi activity.

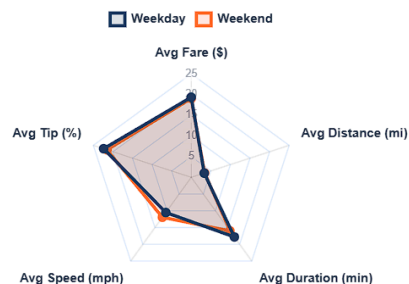


Figure 3: Radar chart — weekday vs weekend across fare, distance, duration, speed, tip

5. Reflection and Future Work

Challenges

The initial ETL pipeline ran in 13 minutes on the full dataset. Profiling identified nine bottlenecks including unnecessary timestamp conversion and CSV-based exclusion logging. Replacing these with NumPy operations and Parquet I/O reduced runtime to approximately 3 minutes. The key lesson was to profile before optimising.

Inserting 33.8 million rows into SQLite was slow in a single process. The solution was multiprocessing using Pool.imap() to process 1-million-row batches in parallel. Multiprocessing was chosen over threading because Python's Global Interpreter Lock prevents threads from executing CPU-bound code in parallel. Processes do not share this limitation. With synchronous mode set to OFF during bulk load, insertion completed in approximately 20 minutes.

API response times on the live database reached 201 seconds for summary queries. The pre-computation approach described in Section 2 resolved this completely by writing all aggregation results to summary tables once, reducing static endpoint response times to under 0.1 seconds.

AI Usage

AI assistance was used to understand how Python's multiprocessing module works, specifically how Pool.imap() distributes batches across worker processes and returns results to the main process. This understanding directly informed the architecture of both cleaner.py and compute_summaries.py.

Future Work

- **PostgreSQL migration:** Would enable parallel query execution and remove the need for pre-computed summary tables for most queries.
- **Docker deployment:** A docker-compose.yml with three services (pipeline, API, frontend) would reduce setup to a single command and make the project portable across machines.
- **Real-time updates:** A scheduled job to download new monthly TLC data, process it through the pipeline, and rebuild summary tables would make the dashboard a live analytics tool.